



INSTITUT UNIVERSITAIRE DE
TECHNOLOGIE VALENCIENNES •
CAMBRAI • MAUBEUGE



SAE S2.03

Documentation Technique

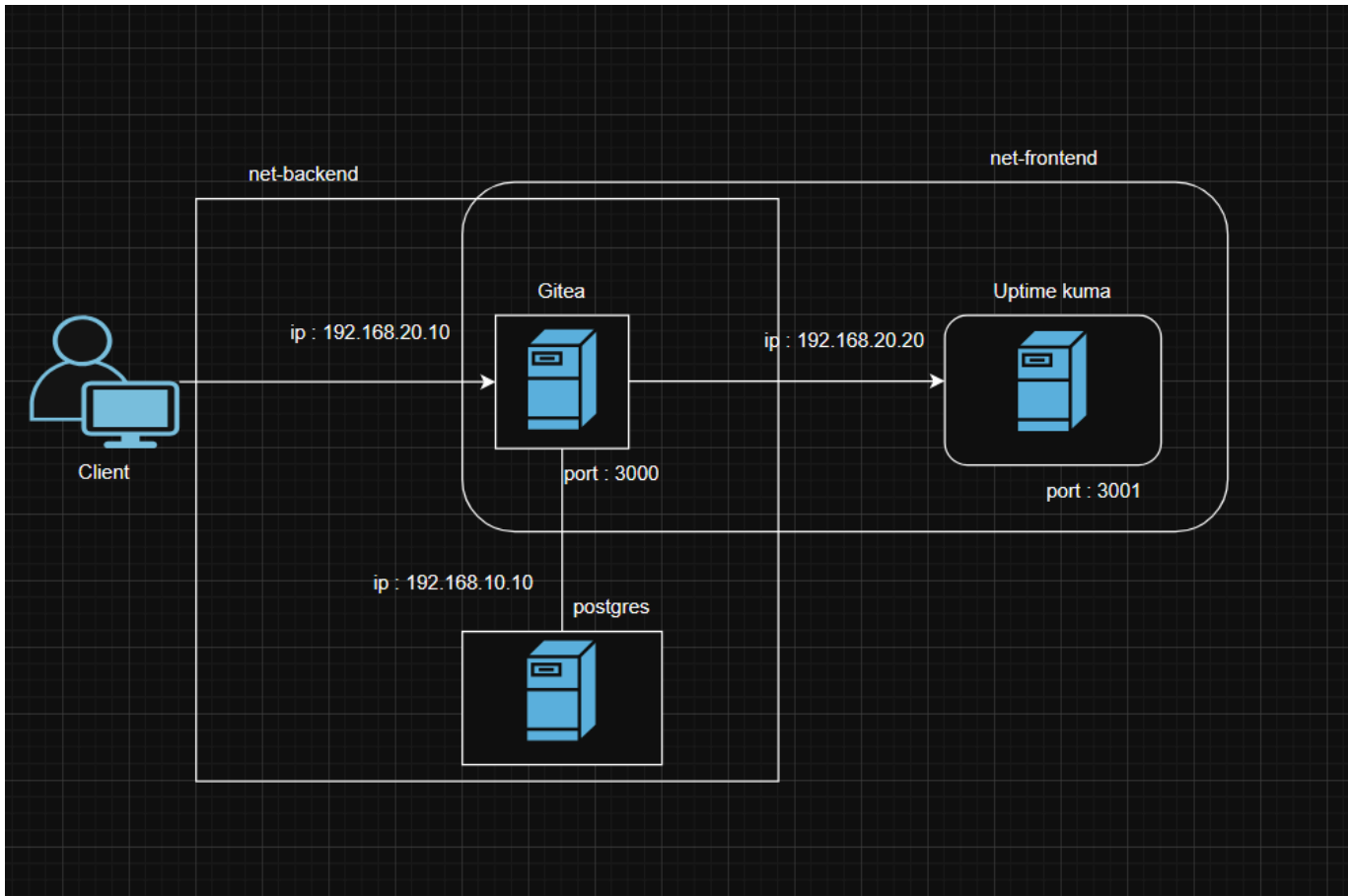
MAYEUX Ounays
EN NILI ZIANI Bilal
BOUDHIM Mehdi

Année universitaire 2025-2026

Sommaire :

Schéma des réseaux Docker :	3
Justification des choix d'isolation / Matrice de flux :	3
1) PostgreSQL sur net-backend uniquement	4
2) Uptime Kuma sur net-frontend uniquement	4
3) Gitea sur les deux réseaux	4
Explication du mécanisme de résolution DNS interne :	5
Qu'est-ce que le DNS interne Docker ?	5
Comment ça fonctionne concrètement	5
Règle fondamentale :	5
Avantages de ce mécanisme :	5
Preuves de Test :	6
Interface Gitea fonctionnelle :	6
Page d'accueil connecté :	6
Dépôt avec fichier pushé :	6
Tableau de bord Uptime Kuma au vert :	7
Uptime Kuma du Gitea avec DNS :	7
Uptime Kuma accessible depuis un autre appareil autre que la machine hôte :	7
Preuve de l'inaccessibilité de PostgreSQL depuis l'hôte/extérieur	8
Avec la commande curl :	8
Avec nmap :	8
Test du git (clonage et push) :	9
Insertion des commandes :	9
Capture d'écran sur le site Gitea montrant que le push a bien été fait :	9
Conclusion :	10

Schéma des réseaux Docker :



Justification des choix d'isolation / Matrice de flux :

Source \ Destination	postgres (192.168.10.10)	Gitea (.10.20 back / .20.10 front)	uptime-kuma (192.168.20.20)	Hôte / Extérieur
postgres (192.168.10.10)	–	OUI – via net-backend	NON – Réseaux différents	NON – Port non mappé
Gitea (.10.20 back / .20.10 front)	OUI – via net-backend	–	OUI - via net-frontend	OUI – :3000 / :2222
uptime-kuma (192.168.20.20)	NON – Réseaux différents	OUI – via net-frontend	–	OUI – :3001
Hôte / Extérieur	NON – Port non mappé	OUI – :3000 / :2222	OUI – :3001	–

1) PostgreSQL sur net-backend uniquement

PostgreSQL est connecté **exclusivement** à *net-backend* et son port 5432 n'est **pas mappé** sur l'hôte.

Justification :

- La base de données contient les données sensibles de la forge (utilisateurs, dépôts etc.). Elle ne doit jamais être exposée directement.
- Seul Gitea a besoin d'interroger PostgreSQL donc aucun autre service n'a de raison légitime d'y accéder.
- L'absence de mapping de port (*5432:5432*) garantit qu'aucune connexion depuis l'hôte ou l'extérieur n'est possible.

Risque évité : Une attaque directe sur la BDD et donc un vol de données sensibles

2) Uptime Kuma sur net-frontend uniquement

Uptime Kuma est connecté **exclusivement** à *net-frontend*, sans aucun accès à *net-backend*.

Justification :

- Uptime Kuma n'a besoin que de vérifier la disponibilité HTTP de Gitea (*http://gitea:3000*). Il n'a aucune raison d'atteindre PostgreSQL.
- En l'isolant sur *net-frontend* même si Uptime Kuma était compromis par un attaquant, il lui serait **impossible** d'atteindre la base de données car la protection ne dépend pas d'un mot de passe ou d'une règle logicielle, mais de la topologie réseau elle-même.

Risque évité : Un rebond réseau où un attaquant qui prendrait le contrôle d'Uptime Kuma ne pourrait pas pivoter vers la BDD.

3) Gitea sur les deux réseaux

Gitea est le **seul** conteneur connecté à *net-backend* ET *net-frontend*.

Justification :

- Gitea est le seul service qui a une raison de parler à la fois à PostgreSQL (lecture/écriture des données) et aux clients externes (interface web, SSH Git).
- Il joue le rôle de **passerelle contrôlée** où les clients externes interagissent avec Gitea, mais ne peuvent jamais atteindre PostgreSQL directement.

Risque évité : Accès direct à la BDD depuis le réseau public donc tout accès aux données passe obligatoirement par la couche applicative Gitea.

Explication du mécanisme de résolution DNS interne :

Qu'est-ce que le DNS interne Docker ?

Quand Docker Compose démarre plusieurs services, il crée automatiquement un **serveur DNS interne** (sur l'IP [127.0.0.11](#) dans chaque conteneur). Ce serveur permet à chaque conteneur de résoudre le **nom d'un autre service** en adresse IP, sans aucune configuration manuelle.

Comment ça fonctionne concrètement

Dans notre infrastructure, Gitea doit se connecter à PostgreSQL. Au lieu d'écrire une IP fixe (qui peut changer), on écrit simplement le **nom du service** :

```
GITEA__database__HOST: postgres:5432
```

Règle fondamentale :

Un conteneur ne peut résoudre le nom d'un autre service que s'ils partagent le même réseau.

Résolution	Possibilité ?	Pourquoi ?
gitea → postgres	OUI	Partagent net-backend
uptime-kuma → gitea	OUI	Partagent net-frontend
uptime-kuma → postgres	NON	Aucun réseau commun

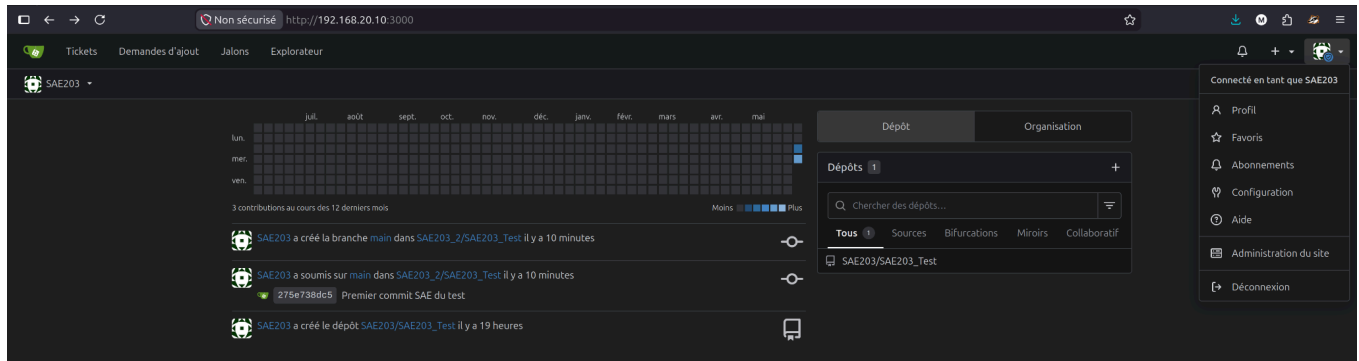
Avantages de ce mécanisme :

- **Pas d'IP à configurer manuellement** : Docker gère l'attribution dynamiquement
- **Portabilité** : le fichier [docker-compose.yml](#) fonctionne sur n'importe quelle machine sans modification
- **Sécurité par la topologie** : un service hors réseau ne peut pas *résoudre* le nom d'un autre service, encore moins s'y connecter

Preuves de Test :

Interface Gitea fonctionnelle :

Page d'accueil connecté :



Dépôt avec fichier pushé :

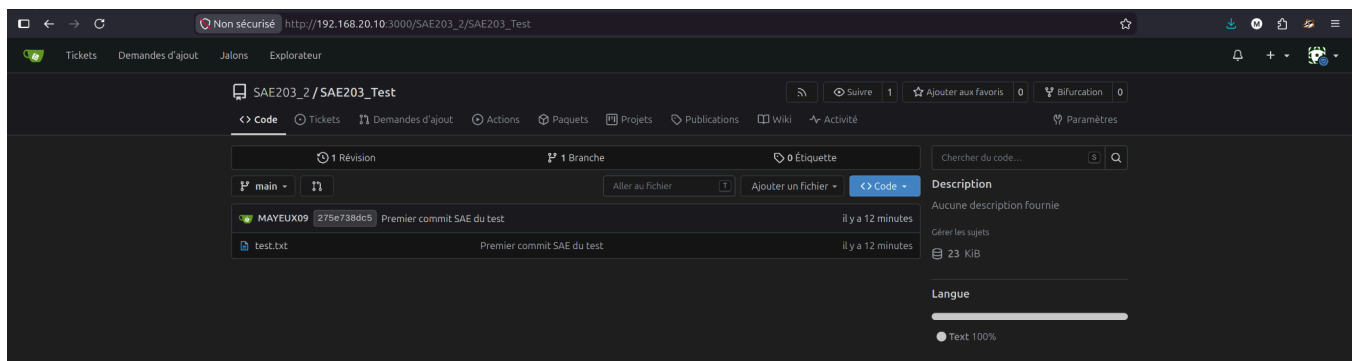
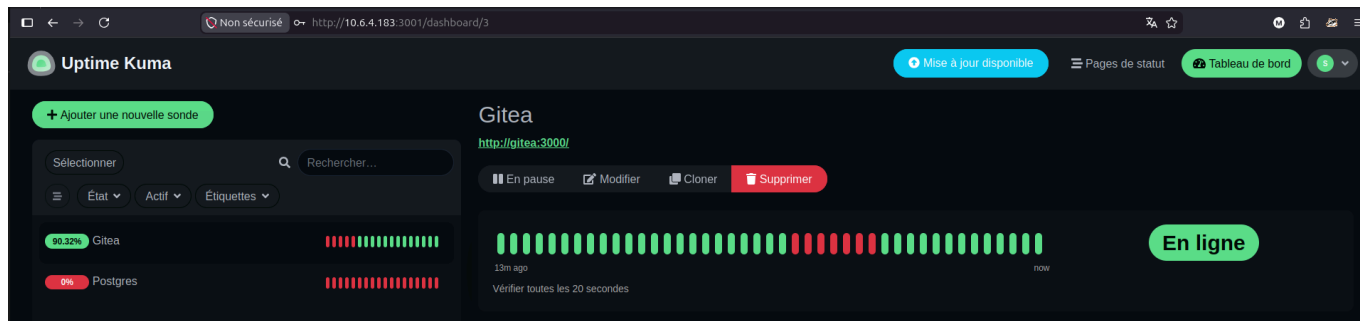
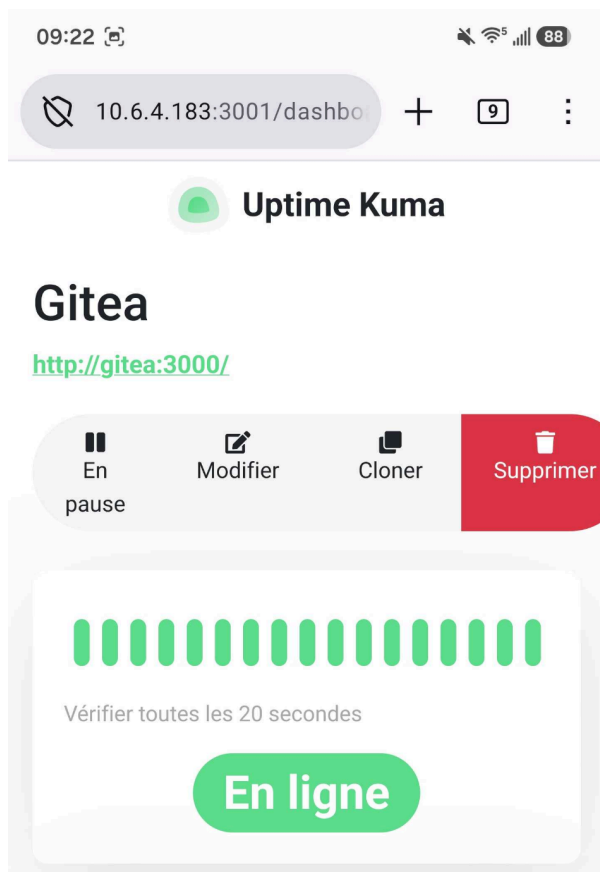


Tableau de bord Uptime Kuma au vert :

Uptime Kuma du Gitea avec DNS (le rouge présent sur l'image est quand j'ai relancé les conteneurs) :



Uptime Kuma accessible depuis un autre appareil autre que la machine hôte :



Preuve de l'inaccessibilité de PostgreSQL depuis l'hôte/extérieur

Avec la commande curl :

Dans l'image ci-dessous, je suis rentré dans le terminal du conteneur Uptime Kuma (avec la commande `docker exec -it sae_uptime_kuma bash`) puis j'ai fait la commande `curl 192.168.10.10:5432` pour vérifier que la base de donnée est bien inaccessible depuis le conteneur d'Uptime Kuma. Comme la réponse à la commande est bien "Connection timed out", on en conclut donc que le conteneur Uptime Kuma n'a pas accès à la base de données.

```
maynux@MAYEUX:~/sae203$ docker exec -it sae_uptime_kuma bash
root@ec6030c27d5b:/app# curl 192.168.10.10:5432
curl: (7) Failed to connect to 192.168.10.10 port 5432: Connection timed out
root@ec6030c27d5b:/app#
```

Avec nmap :

Dans l'image ci-dessous, la commande a été exécutée sur une autre machine personnelle de Ounays qui est sous Windows 11 dans le même réseau que la machine hôte (qui avait à ce moment-là, l'IP 192.168.1.36). On peut voir qu'après son exécution, le terminal renvoie que l'hôte est connecté mais que le port 5432 nommé Postgresql est en état "closed" donc on ne peut pas y accéder

```
PS C:\Users\garfi> nmap -p 5432 192.168.1.36
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-05 16:18 +0200
Nmap scan report for mayeux.home (192.168.1.36)
Host is up (0.12s latency).

PORT      STATE SERVICE
5432/tcp  closed postgresql
MAC Address: 60:E9:AA:F1:40:B5 (Cloud Network Technology Singapore PTE.)

Nmap done: 1 IP address (1 host up) scanned in 0.27 seconds
PS C:\Users\garfi> |
```

Pour montrer que c'est bien ma machine qui a été vérifiée, on va regarder l'adresse MAC sur l'image ci-dessus qu'on va comparer avec la capture d'écran ci-dessous :

```
wlp2s0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.36 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::cfea:d546:8e47:88c3 prefixlen 64 scopeid 0x20<link>
    inet6 2a01:cb0c:f85:2400:fb68:c7e6:ce27:4294 prefixlen 64 scopeid 0x0<global>
    inet6 2a01:cb0c:f85:2400:e605:924c:bcce:1385 prefixlen 64 scopeid 0x0<global>
    ether 60:e9:aa:f1:40:b5 txqueuelen 1000 (Ethernet)
```

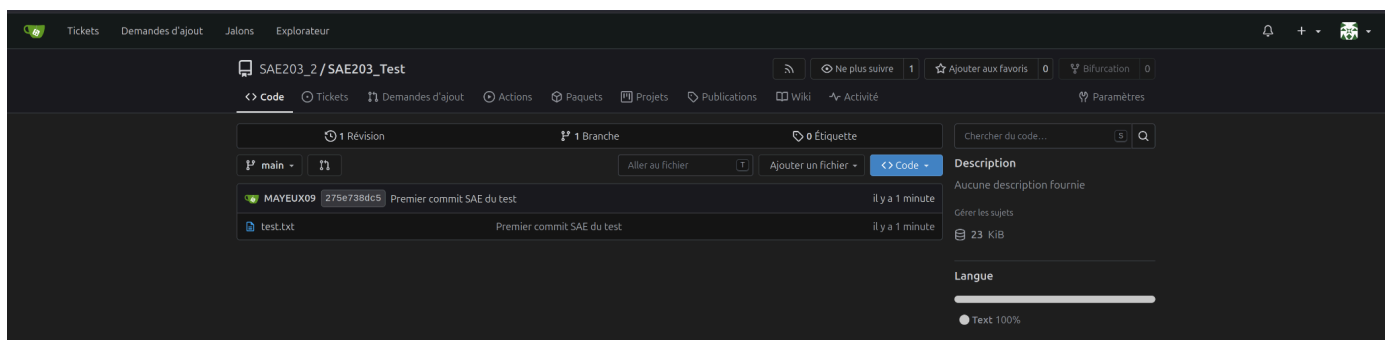

On peut voir que au niveau de la dernière ligne de l'image, l'adresse MAC est 60:e9:aa:f1:40:b5 comme ce qui est aussi indiqué sur la commande Nmap donc c'est bien sur la bonne machine qu'on a vérifié le port qui est donc fermé.

Test du git (clonage et push) :

Insertion des commandes :

```
maynux@MAYEUX:~/sae203$ git clone ssh://git@192.168.20.10:2222/SAE203_2/SAE203_Test.git
fatal: le chemin de destination 'SAE203_Test' existe déjà et n'est pas un répertoire vide.
maynux@MAYEUX:~/sae203$ cd SAE203_Test/
maynux@MAYEUX:~/sae203/SAE203_Test$ echo "Test SAE 203 pour le git" > test.txt
maynux@MAYEUX:~/sae203/SAE203_Test$ git add .
maynux@MAYEUX:~/sae203/SAE203_Test$ git commit -m "Premier commit SAE du test"
[main (commit racine) 275e738] Premier commit SAE du test
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt
maynux@MAYEUX:~/sae203/SAE203_Test$ git push origin main
Énumération des objets: 3, fait.
Décompte des objets: 100% (3/3), fait.
Écriture des objets: 100% (3/3), 255 octets | 255.00 Kio/s, fait.
Total 3 (delta 0), réutilisés 0 (delta 0), réutilisés du pack 0
remote: . Processing 1 references
remote: Processed 1 references in total
To ssh://localhost:2222/SAE203_2/SAE203_Test.git
 * [new branch]      main -> main
maynux@MAYEUX:~/sae203/SAE203_Test$
```

Capture d'écran sur le site Gitea montrant que le push a bien été fait :



Comme vu sur les captures d'écran, les commandes Git SSH sont bien fonctionnels

Conclusion :

Cette SAE nous a permis de concevoir et déployer une forge logicielle complète à partir de zéro, en combinant trois services conteneurisés : PostgreSQL, Gitea et Uptime Kuma qui sont orchestrés via Docker Compose.

Au-delà du simple déploiement, le cœur du travail résidait dans la **sécurisation par la topologie réseau**. En segmentant l'infrastructure en deux réseaux Docker distincts (*net-backend* et *net-frontend*), nous avons appliqué le principe du moindre privilège ce qui veut dire que chaque service n'accède qu'aux ressources strictement nécessaires à son fonctionnement. PostgreSQL reste totalement inaccessible depuis l'extérieur, Uptime Kuma ne peut pas atteindre la base de données et Gitea joue le rôle de passerelle contrôlée entre les deux couches.

Les tests réalisés via vérification *curl* depuis le conteneur Uptime Kuma, scan *nmap* depuis une machine externe, clonage et push SSH via Gitea ont validé concrètement que l'architecture fonctionne comme prévu et que les isolations sont bien effectives.

Ce projet nous a également confronté à des problématiques réelles d'administration système comme la gestion des conflits de ports ou la résolution DNS interne Docker. Ces difficultés, résolues au fil du projet, nous ont montré qu'il était nécessaire et pas simple d'isoler des parties d'un réseau pour garantir qu'une compromission d'un service ne puisse pas se propager à l'ensemble de l'infrastructure.